



تحلیل جامع بازار املاک در پلتفرم دیوار

نام موسسه: مدرسه پردازش و تحلیل داده دقیقه

تهیه کنندگان:

کوروش خدائی نمین

زینب السادات حسینی

ایلیا طاهری نیا

پاییز ۱۴۰۴

فهرست مطالب

۱. مقدمه	۱
۲. معرفی مجموعه داده	۱
ساختار مجموعه داده:	۱
دسته بندی ویژگی ها از نظر مفاهیم کسب و کار:	۱
۳. تحلیل اکتشافی داده (EDA)	۴
شناخت داده ها (INITIAL EXPLORATION)	۴
دسته بندی ویژگی ها براساس نوع داده:	۴
کشف روابط بین ویژگی ها	۵
۴. پاکسازی و آماده سازی داده ها	۶
مدیریت داده های تکراری (HANDLING DUPLICATES)	۶
استانداردسازی نوشتار و نوع داده	۷
۵. خوشه بندی آگهی های املاک	۸
الف - پیش پردازش و آماده سازی داده (PREPROCESSING)	۸
مدیریت مقادیر خالی (Handling Missing Values)	۸
مدیریت مقادیر پرت و اشتباه (Handling outliers)	۱۱
مشکل اول: مقادیر max غیر واقعی و پرت شدید	۱۱
مشکل دوم: مقادیر min مشکوک	۱۲
مشکل سوم: ستون های با واریانس بسیار پایین	۱۲
انتخاب متغیرهای کلیدی (Feature Selection)	۱۲
پیش پردازش نهایی (Scaling and Encoding)	۱۳
ب - اجرای الگوریتم K-MEANS	۱۳
پیدا کردن تعداد بهینه خوشه ها (K)	۱۳
ج - کاهش ابعاد و مصور سازی (VISUALIZATION & DIMENSIONALITY REDUCTION)	۱۴
روش اول: کاهش مستقیم داده ها از ۱۱ به ۲ بعد	۱۴
روش دوم: کاهش داده ها براساس مؤلفه های اصلی	۱۴
۶. مدل سازی قیمت	۱۵

۱۵.....	الف – آماده سازی و پیش پردازش داده ها.....
۱۶.....	استراتژی پاکسازی و پرکردن ستون های عددی.....
۱۷.....	استراتژی پاکسازی و پرکردن ستون های عددی.....
۱۹.....	انتخاب ویژگی ها.....
۱۹.....	مهندسی ویژگی (Feature Engineering).....
۲۰.....	تقسیم داده (Data Splitting).....
۲۱.....	ب – مدل سازی، ارزیابی پایه و بهینه سازی ابرپارامترها (MODELING AND HYPERPARAMETER TUNING).....
۲۱.....	مدل های پایه با کل داده.....
۲۱.....	اضافه شدن Decision Tree.....
۲۲.....	تنظیم فراپارامترها با RandomizedSearchCV.....
۲۴.....	تعریف دسته های قیمتی.....
۲۵.....	۷. دسته بندی (CLASSIFICATION) با استفاده از مقدارهای متنی.....
۲۶.....	الف – آماده سازی مجموعه داده.....
۲۶.....	پاکسازی متون و آماده سازی اولیه.....
۲۶.....	پیش پردازش پیشرفته متون فارسی.....
۲۷.....	آماده سازی نهایی و خروجی داده ها.....
۲۸.....	ب – مدل سازی و دسته بندی.....
۲۸.....	بارگذاری داده و بررسی اولیه.....
۲۸.....	استخراج ویژگی متنی و برداری سازی.....
۲۹.....	تقسیم داده و آماده سازی برای آموزش.....
۲۹.....	مدل سازی دسته بندی هدف (cat3_slug – نوع ملک).....
۳۱.....	کامپایل و پیکربندی مدل LSTM.....
۳۱.....	آموزش مدل و نظارت بر فرآیند یادگیری.....
۳۱.....	تحلیل نتایج آموزش و ارزیابی عملکرد.....
۳۲.....	نتایج نهایی و دستاوردهای مدل.....
۳۲.....	پیش بینی نوع کاربری آگهی دهنده (user_type).....
۳۴.....	۸. نتیجه گیری.....

۱. مقدمه

این گزارش با هدف شناخت، ارزیابی کیفیت و ویژگی‌های مجموعه داده آگهی املاک (real_estate_ads) که از پلتفرم دیوار جمع آوری شده است تهیه شده است. هدف از بکارگیری از این مجموعه داده کشف مدل‌های ارزیابی قیمت، ابزارهای تحلیل بازار املاک و سیستم‌های پیش‌بینی کننده قیمت املاک می باشد. در این گزارش نتایج انجام مراحل اساسی پروژه‌های علوم داده از آماده سازی و تحلیل اکتشافی تا خوشه بندی، مدل سازی و پردازش زبان طبیعی در محیط پایتون ارائه می گردد.

۲. معرفی مجموعه داده

مجموعه داده آگهی املاک پلتفرم دیوار (real_estate_ads) شامل بیش از یک میلیون آگهی املاک است که در ۶۰ ستون ویژگی تعریف شده است؛ این مجموعه، داده‌های معتبری از بازار املاک که بازتابی زنده از رفتار اقتصادی، قیمت گذاری، ترجیحات کاربران و پویایی‌های بازار مسکن در ایران‌اند را نشان می دهد. ساختار مجموعه داده در سایت هاگینگز توسط تهیه کنندگان آن به صورت زیر تعریف شده است:

ساختار مجموعه داده:

دسته بندی ویژگی ها از نظر مفاهیم کسب و کار:

دسته بندی (Categorization)

این بخش به نوع کلی ملک و دسته بندی آن می پردازد.

- cat2_slug: دسته بندی اصلی براساس نوع آگهی (اجاره مسکونی، فروش تجاری و...)
- cat3_slug: دسته بندی اصلی براساس نوع ملک (ویلا، آپارتمان، زمین و...)

موقعیت مکانی (Location)

- neighborhood_slug, city_slug: شناسه های موقعیت مکانی (اسلاگ شهر و محله).
- location_longitude, location_latitude: مختصات جغرافیایی.
- location_radius: شعاع دقت موقعیت مکانی.

جزئیات آگهی (Listing Details)

- created_at_month: زمان ایجاد آگهی
- user_type: نوع کاربر آگهی دهنده (شخص یا املاک)
- title, description: اطلاعات متنی درباره عنوان آگهی و توضیحات مربوطه

اطلاعات مالی و قیمت گذاری ملک (Financial Information)

- مربوط به اجاره (Rent-related):
 - rent_mode: حالت اجاره (مثلاً ماهانه، سالانه)
 - rent_value: مبلغ اجاره.
 - rent_to_single: آیا به مجرد اجاره داده می شود؟
 - rent_type: نوع اجاره (مثلاً مسکونی، اداری)
- مربوط به قیمت فروش (Price-related):
 - price_mode: حالت قیمت (مثلاً قیمت کل، قیمت هر متر مربع).
 - price_value: مبلغ قیمت کل یا قیمت هر متر.
- مربوط به رهن/ودیعه (Credit-related):
 - credit_mode: حالت ودیعه.
 - credit_value: مبلغ ودیعه/رهن.
 - Transformed values: مقادیر تبدیل شده.

مشخصات ملک (Property Specifications)

- ابعاد ملک (به متر مربع): building_size, land_size به ترتیب متراژ زمین و متراژ بنا (زیربنا).
- مشخصات سند: has_business_deed, deed_type: نوع سند (مثلاً تک برگ، شش دانگ) و اینکه آیا ملک سند تجاری/اداری دارد یا خیر.
- جزئیات ساختار ساختمان: floor, rooms_count, total_floors_count, unit_per_floor شامل: شماره طبقه واحد، تعداد اتاق خواب، تعداد کل طبقات ساختمان و تعداد واحد در هر طبقه.
- is_rebuilt, construction_year: سال ساخت و وضعیت بازسازی.

امکانات و ویژگی‌ها (Amenities and Features)

- انشعابات (Utilities): has_water (آب)، has_electricity (برق)، has_gas (گاز).
- سیستم‌های تهویه (Climate control): has_heating_system (سیستم گرمایشی)، has_cooling_system (سیستم سرمایشی).
- امکانات اصلی (Facilities): has_balcony (بالکن)، has_elevator (آسانسور)، has_warehouse (انباری)، has_parking (پارکینگ).
- امکانات لوکس (Luxury features): has_pool (استخر)، has_jacuzzi (جکوزی)، has_sauna (سونا).
- سایر ویژگی‌ها (Other features): has_security_guard (نگهبان)، has_barbecue (باربیکیو)، building_direction (جهت ساختمان: شمالی، جنوبی و ...)، floor_material (جنس کف: سرامیک، پارکت و ...).

اطلاعات اجاره کوتاه‌مدت (Short-term Rental Information)

- این بخش مخصوص املاکی است که به صورت روزانه اجاره داده می‌شوند (مانند ویلا و سوئیت).
- property_type: نوع موقعیت ملک مثلاً: ساحلی، جنگلی و غیره
 - regular_person_capacity, extra_person_capacity: ظرفیت استاندارد و ظرفیت نفرات اضافه.
 - cost_per_extra_person: هزینه به ازای هر نفر اضافه.
 - تغییرات قیمت‌گذاری (Pricing variations):
 - rent_price_on_regular_days: قیمت اجاره در روزهای عادی.
 - rent_price_on_special_days: قیمت اجاره در روزهای خاص (مثلاً تعطیلات عید).
 - rent_price_at_weekends: قیمت اجاره در آخر هفته‌ها (پنجشنبه و جمعه).

۳. تحلیل اکتشافی داده (EDA)

شناخت داده ها (Initial Exploration)

در نخستین گام‌های اکتشافی، بخش‌هایی از داده به نمایش درآمدند تا شناخت بهتری از ساختار، مقادیر گم‌شده (NaN)، و نوع داده‌ها حاصل شود. مشاهده ردیف‌های نمونه نشان داد داده‌ها مخلوطی از رشته‌های فارسی، انگلیسی، اعداد فارسی و انگلیسی، نمادها، و کاراکترهای خاص هستند. همچنین، ستون‌هایی از قبیل title، description، user_type و ... اغلب نیازمند استانداردسازی و تبدیل فرمت نوشتاری‌اند.

بررسی اولیه با استفاده از متدهای head و info نشان داد که بسیاری از ستون‌ها، به‌ویژه ستون‌های مرتبط با مشخصات کمی ملک (مانند انواع قیمت‌ها و ابعاد ملک)، با درصد بالایی از مقادیر گم‌شده (Missing Values) مواجه هستند. همچنین، تحلیل آماری متغیرهای عددی (با استفاده از describe) وجود نقاط پرت (Outliers) شدید را در متغیرهای قیمت و ابعاد مشخص ساخت که این امر لزوم فیلترسازی و مدیریت دقیق این نقاط را در مراحل بعدی پاکسازی ایجاب می‌کند. بنابراین تصمیم گرفته شد تا انواع اصلی داده‌ها بازبینی و چک شود و پس از دسته‌بندی ستون‌ها براساس نوع داده، انجام کلیه مراحل شناخت چالش‌های مجموعه داده شامل شناسایی مقادیر گم‌شده، تکراری و پرت بصورت گروهی صورت پذیرد. سپس استراتژی‌های مدیریت و برخورد با چالش‌ها به صورت اختصاصی روی هر دسته اعمال شود.

برای انجام این کار، ابتدا مقادیر یونیک هر ستون را مشاهده و براساس داده‌های موجود در هر ستون آنها به دسته‌های زیر تقسیم شدند:

دسته‌بندی ویژگی‌ها براساس نوع داده

گروه ۱: متغیرهای عددی پیوسته (Continuous Numerical) که شامل ۱۶ ستون ویژگی به شرح زیر است:

'rent_value', 'price_value', 'credit_value', 'transformable_credit', 'transformable_rent',
'transformed_credit', 'transformed_rent', 'land_size', 'building_size', 'cost_per_extra_person',
'rent_price_on_regular_days', 'rent_price_on_special_days', 'rent_price_at_weekends',
'location_latitude', 'location_longitude', 'location_radius'

گروه ۲: متغیرهای عددی گسسته (Discrete Numerical) که شامل ۲ ستون ویژگی به شرح زیر است:

'regular_person_capacity', 'extra_person_capacity', 'construction_year', 'unit_per_floor',
'total_floors_count', 'rooms_count', 'floor'

گروه ۳: متغیرهای بولین / دودویی (Boolean / Binary) که شامل ۱۷ ستون ویژگی به شرح زیر است:

'rent_to_single', 'has_business_deed', 'has_balcony', 'has_elevator', 'has_warehouse',
'has_parking', 'is_rebuilt', 'has_water', 'has_electricity', 'has_gas', 'has_security_guard',
'has_barbecue', 'has_pool', 'has_jacuzzi', 'has_sauna', 'rent_credit_transform',
'transformable_price'

گروه ۴: متغیرهای دسته‌بندی (Categorical) که شامل ۱۷ ستون ویژگی به شرح زیر است:

'cat2_slug', 'cat3_slug', 'city_slug', 'neighborhood_slug', 'deed_type', 'building_direction',
'floor_material', 'property_type', 'rent_mode', 'credit_mode', 'price_mode', 'user_type', 'rent_type',
'has_warm_water_provider', 'has_heating_system', 'has_cooling_system', 'has_restroom'

گروه ۵: متغیرهای متنی که شامل ۳ ستون ویژگی به شرح زیر است:

'title', 'description', 'created_at_month'

کشف روابط بین ویژگی‌ها

در این مرحله، جهت بررسی روابط میان متغیرهای عددی، از ماتریس همبستگی و نمودار نقشه حرارتی (Heatmap) استفاده شد. انتظار می‌رفت که میان متغیرهای قیمتی (مانند price_value و credit_value) و متغیرهای مربوط به ابعاد ملک (مانند land_size و building_size) همبستگی مثبت و معناداری مشاهده شود.

با این حال، نتایج اولیه نشان‌دهنده عدم همبستگی یا همبستگی بسیار ضعیف در برخی موارد بود. همچنین در بخش‌هایی از نمودار، به دلیل پراکندگی داده‌ها یا وجود مقادیر نامعتبر، ارتباط معناداری یافت نشد (که در نمودار به صورت نواحی بدون مقدار یا سفید ظاهر شدند). این مسئله لزوم پاکسازی داده‌ها و نحوه محاسبه همبستگی‌ها را نشان داد. پس از انجام مراحل پاکسازی که در بخش بعدی آمده است، نمودار نقشه حرارتی (Heatmap) نتایج زیر را ارائه مشخص کرد:

نکات کلیدی در رابطه با rent_value

• قوی‌ترین همبستگی‌ها: location_latitude (۰,۷۲)، price_value (۰,۶۵) و credit_value

(۰,۵۴) بالاترین همبستگی را با اجاره دارند

- همبستگی‌های متوسط: در building_size (۰,۵۰) ، rooms_count (۰,۴۱) و land_size (۰,۳۳) رابطه متوسطی با اجاره نشان می‌دهند
- همبستگی‌های ضعیف: construction_year (۰,۱۰) ، floor (۰,۱۷) و location_longitude (۰,۰۲) ارتباط کمی با اجاره دارند

نکات کلیدی در رابطه با price_value

- قوی‌ترین همبستگی‌ها: rent_value (۰,۶۵) ، land_size (۰,۶۵) ، building_size (۰,۶۵) و credit_value (۰,۶۰) نشان‌دهنده رابطه قوی با قیمت هستند
- location_latitude (۰,۵۶) و rooms_count (۰,۵۵) نیز همبستگی خوبی با قیمت دارند

روابط درون متغیرها

- building_size و rooms_count همبستگی قوی (۰,۷۶) دارند که منطقی است
- land_size و building_size همبستگی خوب (۰,۶۵) نشان می‌دهند
- total_floors_count و unit_per_floor همبستگی قوی (۰,۵۲) دارند که نشان‌دهنده ساختمان‌های بزرگ‌تر با واحدهای بیشتر است
- location_longitude تقریباً با هیچ متغیری همبستگی قوی ندارد (همه زیر ۰,۱)

۴. پاکسازی و آماده‌سازی داده‌ها

مدیریت داده‌های تکراری (Handling Duplicates)

- در اولین بررسی به نظر رسید که در مجموعه داده، هیچ دو رکورد تکراری وجود ندارد. اما برای اطمینان بیشتر، و طبق بررسی بعمل آمده به دو نکته زیر دقت شد:
- دو ستون 'description', 'title' را که بصورت متن فارسی درج شده اند، اگر در بررسی داده‌های تکراری لحاظ شوند، هیچ رکوردی تکراری پیدا نخواهد شد.

- ستون 'created_at_month' می‌تواند باعث شناسایی نادرست تکراری‌ها شود، چون ممکن است یک ملک با شرایط کاملاً یکسان در تاریخ‌های مختلف ثبت شده باشد. برای مدل‌سازی قیمت و خوشه‌بندی، لازم است این تکرارهای مفهومی حذف شوند و از هر ملک فقط یک نمونه باقی بماند. به همین دلیل، آخرین آگهی نگه داشته می‌شود چون جدیدتر و معتبرتر است.

بنابراین تصمیم گرفته شد که با رویکرد مذکور ابتدا موارد تکراری را شناسای کرده و بعد از حذف مقادیر تکراری مقادیر گمشده را تعیین تکلیف شود. اما در بررسی دقیق‌تر رکوردهای تکراری، مشاهده شد که برخی آگهی‌ها اگرچه از نظر سیستمی تکراری شناسایی می‌شوند، اما در واقع مربوط به املاک متفاوتی هستند. علت این خطا، وجود مقادیر گمشده (NaN) در ویژگی‌های کلیدی مانند قیمت و متراژ بود که باعث می‌شد الگوریتم تشخیص تکرار، رکوردهای نامرتبط را یکسان بپندارد. بدین منظور، استراتژی تفکیک داده‌ها به دو دسته «باکیفیت» و «کم کیفیت» اتخاذ گردید:

۱. **داده‌های باکیفیت (High-Quality Data):** رکوردهایی که در ستون‌های کلیدی و اصلی (مانند متراژ و قیمت) مقدار دارند.
 ۲. **داده‌های کم کیفیت (Low-Quality Data):** رکوردهایی که در این ستون‌های کلیدی NaN هستند و هویت مشخصی ندارند.
- سپس، فرآیند شناسایی تکراری را فقط روی داده‌های باکیفیت اجرا و داده‌های کم کیفیت را کنار گذاشته شد. در نهایت در میان ۹۸۱،۶۷۵ رکورد با کیفیت، تعداد ۸،۷۹۰ رکورد تکراری شناخته و حذف شد.

استانداردسازی نوشتار و نوع داده

تبدیل کاراکترها و اعداد به فرمت یکپارچه و استاندارد برای بسیاری از کاربردهای داده‌کاوی و یادگیری ماشین اهمیت حیاتی دارد. داده‌های خام غالباً به زبان فارسی، اما با کاراکترهای غیراستاندارد، اعداد مختلف (مثلاً «۳» فارسی، «۳» انگلیسی، یا «سه») و نمادهای ویژه ذخیره شده‌اند. این پراکندگی نمایشی می‌تواند برداشت آماری را مختل کند و به مدل‌سازی غیردقیق منجر شود.

۱. براساس مشاهدات انجام شده در مرحله دسته بندی ویژگی‌ها، متوجه شدیم که مقادیر موجود در برخی از **ویژگی‌های عددی** شامل چالش‌هایی از جمله حروف فارسی، اعداد فارسی، دارای علائم و کاراکترهای

غیر عددی هستند. بنابراین داده‌های اشتباه در این ستون‌ها پاکسازی و یا تبدیل شد و در نهایت نوع داده همه ستون‌های عددی از object به int یا float تغییر یافت.

۲. همچنین در ستون‌هایی که ماهیتاً از نوع **دسته‌بندی** بودند ولی بدلیل داشتن مقدار گمشده، از نوع آبجکت نمایش داده می شدند، با اختصاص مقدار not_specified در خانه‌های خالی، به نوع categorical تبدیل شدند.

۳. درخصوص ستون‌های از نوع Boolean نیز، تمامی خانه‌ها بصورت یکپارچه با مقدار True یا False پر شدند.

۵. خوشه‌بندی آگهی‌های املاک

این بخش با هدف انجام تحلیل خوشه‌بندی (Clustering Analysis) روی مجموعه داده پاکسازی شده، اجرا شد. در این بخش مسئله اصلی این بود که بتوانیم املاک موجود را بر اساس ویژگی‌های عددی و نوعی آن‌ها، در گروه‌هایی قرار دهیم که از نظر ساختاری و رفتاری بیشترین شباهت را به یکدیگر داشته باشند. در ابتدا تمرکز روی استفاده از الگوریتم K-Means و یافتن تعداد بهینه خوشه‌ها بود. سپس برای ارزیابی پایداری ساختار خوشه‌ها و همچنین کاهش نویز، کاهش ابعاد با PCA انجام شد. در نهایت هر دو فضای ۱۱ بعدی و ۸ بعدی تحلیل شدند تا ببینیم آیا ساختار خوشه‌بندی پایدار است یا خیر.

الف – پیش‌پردازش و آماده‌سازی داده (Preprocessing)

با توجه به اینکه الگوریتم‌های خوشه‌بندی که بر اساس محاسبات ریاضی (مثل فاصله) کار می‌کنند، در این مرحله تلاش شد تا داده‌ها به یک فرمت کاملاً عددی، تمیز و استاندارد تبدیل شود تا الگوریتم به بهترین شکل عمل کند.

مدیریت مقادیر خالی (Handling Missing Values)

برای مدیریت مقادیر گمشده در مجموعه داده، پس از بررسی درصد خالی بودن ستون‌ها و مشاهده نمودارهای همبستگی، مشخص شد که مقادیر گمشده را براساس نوع معامله یعنی ستون 'cat2_slug' بررسی و مدیریت شود. زیرا خالی بودن مقدار برای ستون‌های مربوط به فروش در معامله‌های رهن و اجاره، بدیهی و اجتناب‌ناپذیر است.

سپس به بررسی اینکه مقادیر گمشده در کدام ستون‌ها در "تمام" سطرها بالاتر از یک حد آستانه (۷۰٪) هستند، پرداخته شد. براساس نتیجه این بررسی، در مورد حذف ستون‌هایی که در تمام انواع معامله مقدار خالی بالای حد آستانه را دارند، تصمیم‌گیری شد.

-has_water -land_size -transformed_rent - transformed_credit - rent_to_single -
has_jacuzzi -has_pool -has_barbecue -has_security_guard -has_gas -has_electricity
property_type -has_sauna -

۱. در خصوص ستون‌هایی که مربوط به **امکانات و لوکس بودن ملک** بودند، با فرض اینکه خالی بودن به معنای نداشتن آن امکانات هست، مقادیر خالی آنها را با False پر شد.

۲. در خصوص ستون‌هایی که از نوع **دسته بندی** بودند، مانند ستون property_type مقدار خالی آن را با not_specified پر شد.

۳. در خصوص **ستون rent_to_single** که نشان‌دهنده‌ی تمایل مالک برای اجاره به مجردها بود، و فقط برای آگهی‌های اجاره معنادار بود؛ درصد زیادی از مقادیر آن گمشده بود. با توجه به اینکه اثر آن بر قیمت، در صورت وجود، بسیار ضعیف‌تر از متغیرهایی مانند مترآژ، منطقه و تعداد اتاق‌هاست، به‌عنوان سیگنال ضعیف و غیرعمومی حذف شد.

۴. در خصوص **ستون land_size** ابتدا وابستگی آن با building_size بررسی شد، مشخص شد ویژگی land_size تنها برای املاک نوع «خانه» و «ویلا» معنا دارد و در مواردی که land_size مقدار دارد، معمولاً building_size نیز وجود دارد. با این حال در برخی ردیف‌های ویلایی هر دو مقدار گمشده بودند. بنابراین تصمیم گرفته شد land_size به‌عنوان ویژگی ساختاری کلیدی برای ویلا و خانه حفظ و ۱۲ ردیف که هر دو فیلد land_size و building_size کاملاً خالی بودند، را حذف شود.

○ اصلاح داده‌های اشتباه: در ۲۷,۴۶۲ ردیف که $building_size > land_size$ بود، این دو مقدار با یکدیگر تعویض شدند تا منطق داده حفظ شود.

۵. در بررسی ستون‌های مربوط به **تبدیل اجاره**، دو گروه ستون جلب توجه می‌کردند:

- ستون‌های «قابل تبدیل» (transformable _)
- ستون‌های «تبدیل شده» (transformed _) همراه با پرچم rent_credit_transform

نتایج نشان داد:

- از میان بیش از ۳۵۲ هزار رکورد واجد شرایط تبدیل، فقط حدود ۷۲ هزار خروجی واقعی داشتند؛ یعنی تقریباً ۸۰٪ بدون دلیل منطقی خروجی نداشتند.
- بررسی دقیق ثابت کرد که ستون‌های transformable_ هیچ تغییری ایجاد نمی‌کنند و دقیقاً کپی ستون‌های اصلی (value_) هستند، پس کاملاً زائدند.
- تحلیل ستون‌های transformed_ نیز نشان داد تفاوت مقادیر پر/خالی ناشی از نوع آگهی یا رفتار قیمت نبود؛ بلکه این ستون‌ها مربوط به «فیلتر جستجوی کاربر» هستند، نه ویژگی واقعی ملک. بنابراین وجود آن‌ها در مدل‌سازی باعث نویز و نشت داده می‌شود.

نتیجه نهایی: تمام ستون‌های زیر باید حذف شوند چون یا تکراری‌اند یا رفتاری و غیرقابل استفاده در مدل‌سازی قیمت:

- transformable_credit
- transformable_rent
- transformed_credit
- transformed_rent
- rent_credit_transform

۶. در خصوص ستون **construction_year** که در بخشی از داده‌ها سال ساخت وجود نداشت. مقدار گم‌شده را با صدک ۲۵ام (Q1) کل داده جایگزین کردم تا فرض شود که ملک قدیمی است ولی با داده‌های واقعی هم‌تراز بماند.

۷. در خصوص ستون **building_size** با توجه به اینکه متر از وابستگی شدیدی با تعداد اتاق داشت ابتدا برای رکوردهایی که rooms_count ندارند، مقدار آن را بر اساس میانه‌ی متر از املاک مشابه پر شد. سپس با استفاده از داده‌های تکمیل‌شده‌ی rooms_count، مقادیر گم‌شده‌ی building_size را بر اساس میانه‌ی گروه پر کردم. این فرآیند تا زمانی تکرار شد که هر دو ستون تقریباً بدون مقدار Null شدند.

مدیریت مقادیر پرت و اشتباه (Handling outliers)

برای شناسایی مقادیر پرت، اولین قدم، اجرای دستور describe روی دیتافریم بود. خروجی دستور فوق که در قالب جدول آماری نمایش داده شد، اطلاعات بسیار مهمی درباره وضعیت ستون‌ها و چند مشکل بسیار بزرگ را آشکار کرد.

مشکل اول: مقادیر max غیر واقعی و پرت شدید

در بسیاری از ستون‌ها، مقدار max کاملاً غیر منطقی و مشخص بود که کاربر عمداً داده اشتباهی وارد کرده است:

```
price_value = 999,999,999,999,999
```

```
rent_value = 9,999,999,999,999
```

```
land_size = 10,000,000
```

```
building_size = 10,000,000
```

این اعداد واقعی نبودند. این‌ها مقادیر Placeholder یا خطای سیستم/ورود داده بودند و وجود حتی چند مقدار پرت، کل توزیع را تخریب می‌کند، باعث چولگی شدید می‌شود و نمودارها را بی‌فایده می‌کند. از همه مهم‌تر، هر مدلی مثل K-Means را کاملاً نابود می‌کند. برای رفع مشکل داده‌های پرت در این ستون‌ها چند راهکار وجود داشت:

- حذف رکوردهایی که شامل اینگونه داده‌های پرت هستند.
- جایگزینی این داده‌ها با مقدار آستانه که برای انتخاب این مقدار آستانه دو راهکار وجود دارد:
 - جایگزینی با یک مقداری که از نظر بیزینس قابل قبول باشد، مثلاً ۳ هزار متر برای زمین
 - جایگزینی با مقادیر چارک‌ها

ابتدا نسبت به حذف این رکوردهای که داده‌های کاملاً پرت دارند، اقدام شد.

سپس برای مقادیر پرتی که ناشی از اشتباه کاربر نبوده تصمیم به جایگزینی با یک مقدار آستانه شد. برای انتخاب حد آستانه برای شناسایی مقادیر پرت از چارک‌ها استفاده شد زیرا به جای تعیین آستانه‌های دستی و ذهنی، این آستانه‌ها را بر اساس توزیع واقعی خود داده‌ها محاسبه می‌کند.

متوجه شدیم که به دلیل اینکه ستون‌های قیمت و متراژ تقریباً همیشه دارای چولگی به راست (Right-Skewed) هستند (یعنی تعداد کمی ملک بسیار گران وجود دارد که دُم توزیع را به سمت راست می‌کشد). در این حالت، روش IQR ممکن است بیش از حد سخت‌گیرانه عمل کند. یک تکنیک بسیار رایج، اعمال تبدیل لگاریتمی ($\log_{10}(np)$) روی داده‌ها قبل از محاسبه IQR و سپس برگرداندن آن به مقیاس اصلی است.

پس از آن داده‌های پرت براساس مقادیر مرزی محاسبه شده و توسط چارکها حذف شد. ناگهان مشخص شد مقدار قابل توجهی رکورد حذف شده است. با بررسی‌های بعمل آمده متوجه شدیم به دلیل وجود پارامترهایی مانند `landing_size` نوع ملک در تشخیص داده‌های پرت تأثیرگذار است. بنابراین مراحل بالا را مجدداً براساس نوع ملک اجرا شد.

مشکل دوم: مقادیر min مشکوک

در بعضی از ستون‌ها مقادیر از نظر مفهومی ممکن است اشتباه باشد، مانند `floor = -1` و `rooms_count = 0`. این اعداد ممکن است واقعی باشند یا خطا. بنابراین این مقادیر نیاز به بررسی تکمیلی داشتند اما جزء خطرناک‌ترین مشکلات نبودند. پس از بررسی مشخص شد که این مقادیر مشکلی ایجاد نمی‌کنند و دست نخورده باقی ماندند.

مشکل سوم: ستون‌های با واریانس بسیار پایین

`location_radius` بیشتر مقادیر برابر ۵۰۰ بودند که اطلاعات مفید ندارد. و تصمیم گرفته شد که ستون‌هایی با تنوع بسیار کم حذف یا از مدل خارج شدند.

در پایان برای اطمینان از حل مشکل داده‌های پرت نمودار هیستوگرام مربوط به ویژگی‌های قیمتی و ویژگی‌های متراژی را بررسی و مشخص شد که فرآیند مدیریت داده‌های پرت با موفقیت انجام شد.

انتخاب متغیرهای کلیدی (Feature Selection)

مجموعه داده اولیه شامل ۵۴ ویژگی بود. بسیاری از این ویژگی‌ها یا مقدارهای خالی بسیار زیادی داشتند یا ارتباط مستقیمی با ماهیت ملک نداشتند و یا اطلاعات آن‌ها در متغیرهای دیگر تکرار شده بود. استفاده از همه این ویژگی‌ها باعث می‌شد خوشه‌بندی کیفیت پایینی داشته باشد و سببی برای افزایش نویز باشد.

براساس آنچه در مرحله‌های شناخت مجموعه داده و تحلیل اکتشافی انجام شد، کدام ویژگی‌هایی که واقعاً در تعریف “ماهیت یک ملک” نقش مهمی دارند، انتخاب شد:

- مشخصات فیزیکی ملک مثل `rooms_count`, `building_size`, `landing_size`
- `construction_year`
- امکانات رفاهی مهم مثل `has_elevator`, `has_warehouse`, `has_parking`
- ویژگی‌های مالی مثل `credit_value`, `rent_value`, `price_value`

پیش‌پردازش نهایی (Scaling and Encoding)

با توجه به اینکه الگوریتم‌های خوشه‌بندی مثل K-Means بر اساس “فاصله” بین نقاط کار می‌کنند، اگر مقیاس متغیرها خیلی متفاوت باشد (مثلاً `price_value` در مقیاس میلیارد و `rooms_count` در مقیاس ۱ تا ۵)، متغیری که مقادیر بزرگتری دارد، تاثیر بسیار بیشتری روی محاسبه فاصله گذاشته و نتایج رو خراب می‌کند. بنابراین برای حل این مشکل، متغیرهای عددی را استانداردسازی (Standardization) شد. در این روش، میانگین هر متغیر صفر و انحراف معیار آن یک می‌شود. برای این کار از `StandardScaler` استفاده شد. همچنین متغیرهای بولین (False/True) چون به صورت ۰ و ۱ قابل استفاده هستند، در این مرحله نیازی به تغییر نداشتند.

ب – اجرای الگوریتم K-Means

پیدا کردن تعداد بهینه خوشه‌ها (K)

با داشتن داده‌های آماده، روش Elbow اجرا شد تا روند تغییر مجموع مجذورات درون خوشه‌ای (Inertia) برای مقادیر مختلف K مشاهده شود. با بررسی نمودار، دو نقطه شکست عمده دیده شد و چند مقدار قابل بررسی پیدا شد. به منظور تحلیل دقیق‌تر، ماتریس تقاطع بین برچسب‌های مدل $K=5$ و $K=6$ محاسبه شد. نتیجه نشان داد که $K=6$ توانسته یک خوشه مبهم در $K=5$ را به دو خوشه تفکیک پذیر تقسیم کند:

- یکی شامل ویلاهای اجاره‌ای
- دیگری شامل آپارتمان‌های اجاره‌ای

این موضوع از منظر تفسیرپذیری بسیار ارزشمند بود و نشان داد که افزایش K می‌تواند ساختار واقعی‌تری را در داده‌ها آشکار کند.

سپس ماتریس تقاطع بین برچسب‌های مدل $K=7$ و $K=8$ محاسبه شد. مقایسه خوشه‌ها در قالب ماتریس تقاطع نشان داد که مدل $K=8$ دقت تفکیک را به طرز محسوسی افزایش داده است. خوشه ترکیبی موجود در $K=7$ که شامل املاک تجاری و ویلاهای بزرگ بود، در $K=8$ به دو خوشه متمایز تقسیم شد:

- خوشه ویلاهای اجاره‌ای بزرگ با `land_size` بالا
- خوشه آپارتمان‌های اجاره‌ای با `land_size` کمتر

همچنین خوشه جدیدی شناسایی شد شامل املاک فقط برای فروش با `building_size` بسیار بزرگ و `price_value` بالا؛ گروهی که در مدل‌های قبلی پنهان مانده بود. با توجه به وضوح خوشه‌ها، تفسیرپذیری بالا و کشف گروه جدیدی از املاک تجاری-فروش خاص، $K=8$ به عنوان مدل نهایی انتخاب شد.

ج – کاهش ابعاد و مصورسازی (Visualization & Dimensionality Reduction)

هدف از انجام این مرحله کاهش نویز، افزایش سرعت، بررسی پایداری ساختار خوشه‌ها و ایجاد فضای مناسب‌تر برای مصورسازی بود.

روش اول: کاهش مستقیم داده‌ها از ۱۱ به ۲ بعد

در این روش از ابتدا داده‌های ۱۱ بعدی را مستقیماً در تنها دو محور خلاصه شد. براساس نتایج مشخص شد حجم عظیمی از اطلاعات است (فقط ۴۲,۷۳٪ واریانس حفظ شده) از دست می‌رود. خوشه‌هایی که در فضای ۱۱ بعدی کاملاً از هم مجزا بودند (مانند ملک تجاری و سوئیت اجاره‌ای)، در این فضای ۲ بعدی فشرده، روی هم می‌افتند و دیگر قابل تفکیک نیستند. این نمودار نشان می‌دهد که خوشه‌بندی ضعیف است، ولی در واقع مشکل از روش مصورسازی است.

روش دوم: کاهش داده‌ها براساس مؤلفه‌های اصلی

با رسم نمودار واریانس تجمعی، به این نتیجه رسیدیم که ۸ مؤلفه نخست ۹۰,۲۲٪ اطلاعات را پوشش می‌دهند. بنابراین خوشه‌بندی را در فضای ۸ بعدی انجام شد و با بررسی Elbow جدید مشخص شد که در این فضای جدید نیز همان $K=8$ را پیشنهاد بهترین انتخاب است.

در مقایسه خوشه‌های اصلی با خوشه‌های حاصل از PCA مشخص شد که

- خوشه‌های ۰، ۱، ۲، ۴، ۵، ۶، ۷ همگی بین ۹۶٪ تا ۱۰۰٪ پایداری داشتند.
- خوشه ۳ تنها خوشه‌ای بود که ۱۵,۶٪ اعضایش به خوشه ۲ منتقل شد.

این دو خوشه در فضای اصلی نیز شباهت‌هایی داشتند و PCA با حذف سه مؤلفه کم‌اهمیت توانست این هم‌پوشانی را واضح‌تر نشان دهد. پس در پایان ساختار اصلی خوشه‌بندی تقریباً به طور کامل حفظ شده و PCA هیچ تداخلی با ساختار خوشه‌ها ایجاد نکرده است.

۶. مدلسازی قیمت

مدلسازی قیمت ملک در تهران تنها یک مسئله پیش‌بینی ساده نیست، بلکه به شدت به کیفیت و عمق داده‌های ورودی وابسته است. داده‌های خام آگهی‌ها معمولاً مملو از مقادیر گمشده، ناسازگاری‌های محتوایی، فرمت‌های متفاوت و تکرارهای مفهومی هستند؛ عواملی که در صورت عدم اصلاح، مدل‌های رگرسیونی را دچار خطا و نوسان می‌کنند. به همین دلیل، پیش از مدل‌سازی، یک مرحله پاکسازی جامع شامل استانداردسازی مقادیر، حذف پرت‌ها، جایگزینی هوشمند بر اساس گروه‌بندی، تبدیل انواع داده، و مهندسی ویژگی‌های کلیدی انجام شد تا داده‌ها به شکلی پایدار و قابل تحلیل آماده شوند.

در این فصل پس از مرور این آماده‌سازی، مجموعه‌ای از مدل‌های رگرسیونی شامل Linear Regression، K-Nearest Neighbors (KNN)، Decision Tree Regressor و Multi-Layer Perceptron (MLP) ارزیابی و مقایسه می‌شوند. سپس با بهینه‌سازی ابرپارامترها، بهترین مدل انتخاب شده و تحلیل نهایی قیمت‌ها ارائه می‌گردد تا تصویری دقیق، قابل‌اتکا و مبتنی بر داده از وضعیت بازار مسکن تهران به دست آید.

الف – آماده سازی و پیش‌پردازش داده‌ها

بررسی ساختار مجموعه داده مربوط به تهران نشان داد که به طور میانگین ۵۶,۳۰٪ از کل مقادیر مجموعه داده خالی (NaN) هستند. این میزان بالا عمدتاً ناشی از وجود ستون‌هایی است که برای بخش بزرگی از آگهی‌ها اطلاعات ندارند؛ مانند ویژگی‌های مرتبط با اجاره روزانه یا ستون‌هایی که تقریباً کاملاً خالی هستند (مثل rent_to_single و property_type). این بررسی تأکید کرد که لازم است تمرکز پاکسازی بر ستون‌های کلیدی مرتبط با قیمت ملک، ابعاد فیزیکی و موقعیت جغرافیایی قرار گیرد.

استراتژی پاکسازی و پر کردن ستون‌های عددی

برای هر ستون عددی مهم، یک روش سه مرحله‌ای اعمال شد: اولاً حذف مقادیر پرت (Outliers) و غیر واقعی برای بهبود کیفیت توزیع، ثانیاً پر کردن مقادیر گمشده با استفاده از روش‌های مبتنی بر گروه‌بندی (Group-wise Imputation) برای حفظ زمینه (Context) اطلاعات، و ثالثاً اعمال یک فالبک (Fallback) سراسری در صورت باقی ماندن مقادیر خالی. پاکسازی ستون‌های مالی (price_value, credit_value, rent_value)

۱. پاکسازی ستون‌های مالی (price_value, credit_value, rent_value)

- برای rent_value، مقادیر کمتر از ۵۰۰ هزار تومان و بیشتر از ۲۰۰ میلیون تومان به عنوان پرت علامت گذاری و حذف شدند. سپس تمامی مقادیر خالی با میانه ستون جایگزین شدند.
- برای credit_value نیز آستانه‌های مطلق ۵۰ میلیون و ۵۰ میلیارد تومان اعمال و سپس مقادیر گمشده با میانه پر شد.
- در مورد price_value، علاوه بر سقف مطلق ۱۰۰ میلیارد تومان، از IQR Fence به صورت گروه‌بندی شده بر اساس cat3_slug استفاده شد تا پرت‌ها متناسب با نوع ملک حذف شوند.
- در این مرحله price_value به دلیل نقش آن به عنوان متغیر هدف هنوز جایگزینی نشد و حدود ۴۷,۲۶٪ داده‌های آن خالی باقی ماند.

۲. پاکسازی ابعاد فیزیکی (land_size و building_size)

- building_size که تنها ۲,۲۰٪ داده گمشده داشت، پس از حذف مقادیر کمتر از ۲۰ و بیشتر از ۱۵۰۰ مترمربع، با رویکرد سلسله‌مراتبی جایگزینی شد:
 - میانه محله (neighborhood_slug) برای محله‌های با حداقل ۲۰ داده
 - میانه دسته معاملاتی (cat2_slug)
 - و در نهایت این ستون کاملاً تکمیل شد.
- land_size که گمشده بالایی داشت، ابتدا پاکسازی شد و سپس با میانه cat3_slug و بعد میانه کلی پر گردید تا ستون کاملاً تکمیل شود.

۳. پاکسازی ویژگی‌های ساختمانی (rooms_count و total_floors_count، floor)

- برای total_floors_count، مقادیر زیر ۱ یا بالای ۶۰ حذف شدند. همچنین اگر مقدار floor از total_floors_count بزرگ‌تر بود، هر دو مقدار حذف شدند. سپس جایگزینی با میانه cat3_slug و میانه کلی انجام شد.
- در ستون floor نیز پس از اعمال قوانین سازگاری و حذف مقادیر غیرمنطقی (کمتر از -۱ یا بیشتر از ۵۰)، جایگزینی مبتنی بر slug_cat و total_floors_count انجام شد. با این حال، همچنان حدود ۱۵,۵٪ مقادیر خالی باقی ماند که نشان می‌دهد اطلاعات طبقه برای بسیاری از آگهی‌ها موجود نبوده است.
- برای rooms_count، مقدار صفر به عنوان مقدار نامعتبر به NaN تبدیل و سپس با میانه slug_cat پر شد.

۴. پاکسازی موقعیت جغرافیایی (location_latitude و location_longitude)

- این دو ستون حدود ۲۵٪ مقدار خالی داشتند.
- آگهی‌هایی که فقط یکی از دو مختصات را داشتند حذف شدند، زیرا مختصات باید جفت باشند.
- جایگزینی با میانه محله برای محله‌های با حداقل ۲۰ رکورد انجام شد.
- پس از این مرحله تعداد مقادیر خالی به ۳۲ مورد (۰,۰۲٪) کاهش یافت.
- این مرحله پاکسازی نهایی، تعداد رکوردهای قابل استفاده را به ۸۴,۵۸۶ ملک کاهش داد که آماده مهندسی ویژگی و مدل‌سازی شدند.

استراتژی پاکسازی و پر کردن ستون‌های عددی

پس از تکمیل ستون‌های عددی، اولین قدم در این نوت‌بوک، شناسایی ستون‌های با نوع داده‌ی غیر عددی (object، bool، یا category) بود. این ستون‌ها به دو گروه اصلی تقسیم شدند: ستون‌های دسته‌بندی با تعداد محدود مقادیر منحصر به فرد، و ستون‌های دودویی (که با پیشوند _has مشخص شده و نشان‌دهنده‌ی امکانات ملک هستند). هدف اصلی، پر کردن مقادیر گمشده (NaN) در این ستون‌ها به روشی بود که کمترین سوگیری را به تحلیل‌های بعدی تحمیل کند.

۱. پرکردن ستون‌های دسته‌بندی اصلی

ستون‌های دسته‌بندی اصلی که حاوی اطلاعات مهمی درباره نوع کاربری و وضعیت آگهی بودند، به ترتیب با استراتژی‌های زیر تکمیل شدند:

- **ستون user_type (نوع کاربر):** این ستون نوع آگهی‌دهنده (شخصی یا آژانس) را مشخص می‌کند. به دلیل اهمیت این ویژگی در قیمت‌گذاری و مدل‌سازی، مقادیر گمشده‌ی آن با استفاده از **شایع‌ترین مقدار (Mode)** در کل مجموعه داده پر شدند. این روش، با فرض اینکه داده‌های گمشده از همان توزیع داده‌های مشاهده شده پیروی می‌کنند، برای متغیرهای دسته‌بندی دوحالتی (Binary Categorical) یک رویکرد استاندارد است.
- **ستون‌های cat1_slug, cat2_slug, و cat3_slug (دسته‌بندی ملک):** این ستون‌ها سلسله‌مراتب نوع ملک را از کلی‌ترین (cat1) تا جزئی‌ترین (cat3) نشان می‌دهند. در این ستون‌ها، مقادیر گمشده با مقدار **missing** پر شدند. دلیل این انتخاب، جلوگیری از سوگیری ناشی از استفاده از شایع‌ترین مقدار و نگهداری اطلاعات مربوط به عدم وجود دسته‌بندی در مدل‌سازی بود. با این کار، **missing** خود به عنوان یک دسته جدید و معنادار برای آگهی‌های با نوع ملک نامشخص، در مرحله‌ی One-Hot Encoding لحاظ خواهد شد.
- **ستون status (وضعیت ملک):** وضعیت آگهی، مانند «در حال ساخت»، «نوساز»، یا «قدیمی»، در این ستون ثبت شده بود. مقادیر گمشده در این ستون نیز با **شایع‌ترین مقدار (Mode)** پر شدند، زیرا فرض بر این بود که در اغلب موارد، آگهی‌های فاقد این ویژگی، از وضعیت غالب در بازار پیروی می‌کنند و استفاده از یک مقدار پرت برای این ستون‌های متنی می‌تواند توزیع را دچار انحراف سازد.

۲. پرکردن ستون‌های ویژگی‌های دودویی (Boolean Features)

بخش بزرگی از ستون‌های غیرعددی شامل ویژگی‌های امکاناتی بودند که با پیشوند **_has** (مانند **has_elevator**، **has_storage**، **has_parking**) مشخص شده و ماهیت **دودویی** (True/False) یا وجود/عدم وجود داشتند. درصد بالایی از مقادیر گمشده در این ستون‌ها (نزدیک به ۱۰۰٪ در برخی موارد) نشان می‌دهد که اگر یک ویژگی در آگهی ذکر نشده باشد، به احتمال زیاد آن ملک فاقد آن امکانات است یا آن ویژگی به طور خاص برای آگهی‌دهنده مهم نبوده تا آن را پر کند.

لذا، استراتژی اتخاذ شده برای تمام ستون‌های **_has** به شرح زیر بود:

- **تبدیل نوع داده:** ستون‌هایی که به صورت object ذخیره شده بودند، به نوع داده‌ی دودویی (Boolean) تبدیل شدند (مقادیر غیر خالی به True).

- **جایگزینی با مقدار ثابت:** تمام مقادیر گمشده (NaN) در این ستون‌ها، با مقدار False پر شدند.

این تصمیم بر اساس اصل «سکوت به معنای فقدان» در داده‌های آگهی اتخاذ شد. به عبارت دیگر، عدم ذکر یک امکان مانند آسانسور یا پارکینگ، به‌طور منطقی معادل با عدم وجود آن امکان در نظر گرفته شد. این رویکرد به ویژه برای ستون‌هایی مانند has_separate_daily_rent که تقریباً به‌طور کامل خالی بودند، ضروری بود تا آن‌ها بتوانند به عنوان یک ویژگی دودویی معنادار در مدل استفاده شوند.

انتخاب ویژگی‌ها

به منظور تمرکز پروژه بر مدل‌سازی قیمت فروش املاک شهر تهران، لازم بود تا یک زیرمجموعه از داده‌ها انتخاب شود. بر این اساس، رکورد‌هایی که در دسته‌ی 'residential-sell' (فروش مسکونی) قرار داشتند و ستون هدف، یعنی price_value، در آن‌ها مقدار خالی نداشت، فیلتر شدند. همچنین، ستون‌های نامرتب با حوزه فروش مانند اطلاعات مربوط به رهن و اجاره (rent_mode, rent_value, rent_type, credit_mode و غیره)، ستون‌های متنی آزاد مانند description و title، و ویژگی‌های زمانی غیرضروری مانند created_at_month و location_radius که فاقد اطلاعات ارزشمند برای مدل‌سازی قیمت فروش بودند، از مجموعه ویژگی‌ها حذف گردیدند. این عملیات باعث کاهش حجم مجموعه داده به ۹۴،۵۴۱ رکورد مرتبط با قیمت فروش شد.

مهندسی ویژگی (Feature Engineering)

چندین ویژگی جدید برای افزایش قدرت پیش‌بینی مدل مهندسی شدند:

- **سن بنا (age_of_building):** این ویژگی با کسر سال ساخت ملک (construction_year) از سال

۱۴۰۳ محاسبه شد. سپس ستون construction_year که اکنون تبدیل شده بود، حذف گردید. این ویژگی، فرسودگی و قدمت ملک را به شکلی مستقیم‌تر از صرف سال ساخت به مدل منتقل می‌کند.

- **اثر طبقه بر آسانسور (elevator_floor_effect):** برای مدل‌سازی تعامل بین داشتن آسانسور و موقعیت

واحد در طبقات، یک ویژگی ترکیبی طراحی شد. در این ویژگی، اگر ساختمان دارای آسانسور باشد، مقدار

اثر طبقه برابر با عدد طبقه ضرب در ۱,۵ در نظر گرفته می شود تا افزایش مطلوبیت و ارزش واحدهای طبقات بالاتر در Presence آسانسور بازتاب داده شود. در مقابل، اگر ساختمان فاقد آسانسور باشد، مقدار اثر طبقه برابر با عدد طبقه ضرب در ۰,۷ محاسبه می شود تا کاهش جذابیت و ارزش واحدهای طبقات بالاتر بدون آسانسور به صورت یک وزن دهی کمتر در مدل لحاظ شود.»

- **متوسط قیمت محله (neighborhood_avg_price):** برای در نظر گرفتن تأثیر منطقه بر قیمت، میانه‌ی قیمت ملک (price_value) در هر محله (neighborhood_slug) محاسبه و به عنوان یک ویژگی جدید به هر رکورد اضافه شد. استفاده از میانه، به جای میانگین، به دلیل کاهش تأثیر آگهی‌های با قیمت‌های بسیار پرت در تعیین میانگین محله‌ها صورت گرفت.

در ادامه، جهت افزایش پایداری و عملکرد مدل‌های رگرسیون، خصوصاً در مواجهه با توزیع‌های چوله‌ی قیمت:

- **حذف داده‌های پرت:** داده‌های پرت در متغیر هدف (price_value) با حذف ۱٪ پایین و ۱٪ بالای داده‌ها مدیریت شدند. سپس برای ویژگی‌های اندازه‌ی ملک (building_size و land_size)، روش سختگیرانه‌تر **محدوده‌ی بین چارکی (IQR)** برای حذف داده‌های پرت به کار گرفته شد تا تأثیر مقادیر بسیار غیرعادی در ابعاد ملک بر مدل کاهش یابد.

- **تبدیل لگاریتمی متغیر هدف:** ستون هدف (price_value) با استفاده از لگاریتم طبیعی (np.log) تبدیل شد. این تبدیل توزیع قیمت‌ها را نرمال‌تر کرده، و از تسلط قیمت‌های بسیار بالا در تابع هزینه مدل‌ها جلوگیری می‌کند.

پس از این مراحل، مجموعه داده نهایی شامل ۸۴,۵۸۶ رکورد و ۲۲ ویژگی نهایی برای مدل‌سازی آماده شد.

تقسیم داده (Data Splitting)

برای تضمین ارزیابی معتبر مدل، داده‌ها در ابتدا به دو بخش مجموعه آموزش (Train Set) به میزان ۸۰٪ و مجموعه تست (Test Set) به میزان ۲۰٪ تقسیم شدند. سپس، مجموعه آموزش مجدداً به دو قسمت آموزش (Train) و اعتبارسنجی (Validation) تقسیم شد تا مجموعه اعتبارسنجی به عنوان یک بخش ایزوله‌شده برای انتخاب مدل و تنظیم اولیه ابرپارامترها (Hyperparameter Tuning) پیش از ارزیابی نهایی بر روی مجموعه تست استفاده شود.

برای ورود داده‌ها به مدل‌های یادگیری ماشین، یک پایپ‌لاین پیش‌پردازش (Preprocessing Pipeline) با استفاده از ColumnTransformer تعریف شد:

- ویژگی‌های عددی (numerical_features): با استفاده از StandardScaler استانداردسازی (Standardization) شدند. این عملیات مقادیر را به گونه‌ای مقیاس‌بندی می‌کند که میانگین صفر و انحراف معیار یک داشته باشند، که برای عملکرد بهینه مدل‌هایی نظیر KNN و شبکه‌های عصبی ضروری است.
- ویژگی‌های دسته‌ای (categorical_features): با استفاده از OneHotEncoder کدگذاری (Encoding) شدند. این روش هر دسته (Category) را به یک ستون دودویی (Binary) مجزا تبدیل می‌کند.

ب - مدل‌سازی، ارزیابی پایه و بهینه‌سازی ابرپارامترها (Modeling and Hyperparameter Tuning)

مدل‌های پایه با کل داده

در مرحله‌ی اول، سه مدل پایه Linear Regression، KNN و MLPRegressor روی کل داده‌ی آموزشی (بدون تنظیم فراپارامترها) آموزش داده شده‌اند.

- **Linear Regression**: با وجود سادگی، مقدار $R^2 = 0.77$ روی داده‌ی اعتبارسنجی به دست آورده که نشان می‌دهد بخش قابل قبولی از واریانس قیمت توسط ویژگی‌ها توضیح داده می‌شود.
- **KNN Regressor** با تنظیم دستی اولیه همسایه‌ها و وزن distance: این مدل دقت بهتری از رگرسیون خطی داشته و $R^2 = 0.83$ کسب کرده است؛ بنابراین در تنظیم اولیه، KNN بهترین عملکرد را بین مدل‌های پایه دارد.
- **MLPRegressor** پایه: شبکه عصبی چندلایه بدون تنظیم مناسب فراپارامترها عملکرد ضعیفی داشته و مقدار R^2 منفی به دست آورده که نشان می‌دهد مدل عملاً بدتر از مدل میانگین عمل کرده است.

اضافه شدن Decision Tree

در گام بعدی، به منظور مقایسه بهتر، یک DecisionTreeRegressor ساده نیز به جمع مدل‌ها اضافه شده است.

- **Decision Tree پایه (بدون تنظیم عمق و حداقل نمونه‌ها):** این مدل $R^2 = 0.76$ روی داده‌ی

اعتبارسنجی به دست آورده است؛ دقت آن کمی پایین تر از KNN ولی نزدیک به Linear Regression است و نشان می‌دهد در تنظیم اولیه دچار بیش‌برازش یا کم‌برازش ملایم است.

در این مرحله مشخص می‌شود که:

- از بین مدل‌های پایه، KNN بالاترین دقت (بیشترین R^2) را دارد.

- Linear Regression و Decision Tree در حد متوسط و قابل قبول هستند.

- MLP بدون تنظیم، به دلیل انتخاب نامناسب معماری و فرآپارامترها، دقت بسیار پایینی دارد و نیاز جدی به تنظیم دارد.

تنظیم فرآپارامترها با RandomizedSearchCV

به دلیل محدودیت منابع محاسباتی، برای فرآیند RandomizedSearchCV فقط از زیرمجموعه‌ای شامل حدود ۲۰۰۰ نمونه از داده‌ها استفاده شده تا بتوان بهترین ترکیب فرآپارامترها را برای مدل‌های درخت تصمیم و شبکه عصبی پیدا کرد. این کار باعث کاهش هزینه محاسباتی جست‌وجو، در عین حفظ نمایندگی آماری مناسب از توزیع داده، شده است.

Decision Tree بعد از Random Search

برای DecisionTreeRegressor، فضای جست‌وجو شامل مقادیری برای max_depth و min_samples_split بوده و RandomizedSearchCV با معیار خطای مطلق میانگین (MAE) اجرا شده است.

- **بهترین تنظیم به دست آمده:** عمق درخت در حدود ۱۰ و حداقل نمونه برای انشعاب حدود ۲۰ انتخاب شده است که تعادلی بین پیچیدگی و تعمیم‌پذیری ایجاد می‌کند.

- **دقت مدل بعد از تنظیم:** مقدار $R^2 = 0.77$ روی داده‌ی اعتبارسنجی گزارش شده که نسبت به درخت پایه اندکی بهبود یافته و نشان می‌دهد کنترل عمق و حداقل نمونه‌ها جلوی بیش‌برازش را می‌گیرد و دقت (accuracy) مدل را افزایش می‌دهد.

MLPRegressor بعد از Random Search

برای RandomizedSearchCV، MLPRegressor روی معماری لایه‌های مخفی (hidden_layer_sizes)، تابع فعال‌سازی (activation) و تعداد تکرارها (max_iter) اجرا شده است.

- **بهترین تنظیم حاصل:** معماری با یک لایه‌ی پنهان نسبتاً بزرگ (مثلاً ۶۴ نورون) با فعال‌سازی ReLU و تعداد تکرار حدود ۲۰۰ انتخاب شده است؛ این ترکیب با توجه به داده‌های مسئله توانسته الگوهای غیرخطی را بهتر مدل کند.

- با وجود تنظیم، مقدار R^2 همچنان منفی باقی مانده است که نشان می‌دهد حتی با بهترین فرآیندهای پیدا شده روی زیرمجموعه‌ی ۲۰۰ تایی، شبکه عصبی در این تنظیم خاص نتوانسته تعمیم مناسبی روی داده‌ی اعتبارسنجی داشته باشد و از نظر دقت از سایر مدل‌ها عقب‌تر است.

دلایل اصلی ضعف MLP در این مرحله

در این بخش توضیح داده شده که چرا شبکه عصبی MLP، حتی بعد از چند بار تنظیم، هنوز دقت بسیار پایینی (عدد R^2 منفی و نزدیک صفر) دارد و عملاً فقط میانگین قیمت را پیش‌بینی می‌کند. مهم‌ترین دلایل مطرح شده عبارت‌اند از:

- هم‌خطی و هم‌پوشانی زیاد ویژگی‌ها، معماری نسبتاً ساده نسبت به پیچیدگی الگوهای قیمت، و وجود تعداد زیادی ویژگی تنک (مثلاً OneHot) که یادگیری پایدار را برای شبکه سخت کرده است.
- انتخاب نامناسب هایپرپارامترهای منظم‌ساز (مثل α) و تعداد تکرارها، که باعث شده مدل یا دچار کم‌آموزی شود و یا نتواند به حد کافی روی الگوهای غیرخطی متمرکز شود و در نتیجه دقت (بر حسب R^2) عملاً بهبود پیدا نکند.

در ادامه چند معماری و تنظیم مختلف برای MLP با اندازه‌های لایه مخفی، α و max_iter متفاوت روی همان زیرمجموعه‌ی داده تست شده‌اند، اما خروجی‌ها نشان می‌دهد که در تمام این سناریوها مقدار R^2 همچنان منفی باقی مانده است.

این نتایج نشان می‌دهد که در این مرحله و با این حجم داده و نوع ویژگی‌ها، حتی با تغییرات متوالی در معماری، MLP نمی‌تواند به دقت مدل‌هایی مثل KNN یا Decision Tree نزدیک شود و از نظر accuracy گزینه مناسبی نیست.

تعریف دسته‌های قیمتی

برای تحلیل نهایی، ابتدا نسبت قیمت واقعی به قیمت پیش‌بینی شده برای هر آگهی محاسبه شده است بر اساس این نسبت، سه دسته قیمتی تعریف شده است: اگر price_ratio بیشتر از ۱,۲ ملک گران (over-value)، اگر price_ratio کمتر از ۰,۸ ملک ارزان (under-value) و در بازه بین ۰,۸ تا ۱,۲ ملک مناسب (normal) در نظر گرفته شده است.

۱. توزیع تعداد خانه‌ها در هر دسته

نمودار ستونی نشان می‌دهد که بیشترین تعداد واحدها در دسته قیمت مناسب قرار گرفته‌اند و تعداد کمتری به عنوان گران یا ارزان شناسایی شده‌اند؛ این موضوع با توجه به آموزش مدل رگرسیونی و دقت قابل قبول آن بر روی داده تست قابل انتظار است.

به صورت عددی، حدود ۹ هزار واحد در دسته normal، حدود ۲ هزار واحد در دسته over-value و حدود ۱,۸ هزار واحد در دسته under-value قرار گرفته‌اند که نشان می‌دهد مدل توانسته اکثریت بازار را در محدوده قیمت منصفانه تشخیص دهد.

۲. مقایسه میانگین متراژ و قیمت واقعی در دسته‌ها

میانگین متراژ بنا در سه دسته بسیار نزدیک است؛ برای نمونه، در داده تست متراژ میانگین در دسته normal حدود ۹۸ متر، در over-value حدود ۹۴ متر و در under-value حدود ۹۳ متر است. در مقابل، تفاوت اصلی در قیمت واقعی دیده می‌شود: میانگین قیمت واقعی در دسته normal حدود ۸,۹۹ میلیارد تومان در over-value حدود ۱۰ میلیارد تومان و در under-value حدود ۴,۲۵ میلیارد تومان است. بنابراین با متراژ تقریباً مشابه، خانه‌های گران بیش از حد، تقریباً ۲۰ تا ۳۰ درصد بالاتر از سطح عادی قیمت گذاری شده‌اند و خانه‌های ارزان، تقریباً نصف قیمت مناسب را دارند.

۳. تحلیل مکانی دسته‌های قیمتی

گزارش شمارش محله‌ها نشان می‌دهد که در دسته **قیمت مناسب**، محله‌هایی مثل «پونک»، «جیهون» و «سازمان برنامه» بیشترین تعداد آگهی را دارند که می‌تواند به معنای تعادل نسبی عرضه و تقاضا و قیمت‌گذاری منطقی در این مناطق باشد.

در دسته **گران (over-value)** نیز محله «پونک» در صدر است، همراه با مناطقی مانند «چیتگر-لیک» و «سرسبیل» که احتمالاً به دلیل تقاضای بالا یا دیدگاه فروشندگان، بخشی از فایل‌ها با قیمتی بالاتر از سطح مدل قیمت‌گذاری شده‌اند.

تفسیر کاربردی برای دسته‌های قیمتی

به طور خلاصه، مدل رگرسیونی با دقت مناسب (بر اساس R^2 مراحل قبلی) امکان تفکیک آگهی‌ها به سه سطح قیمتی را فراهم کرده است و نشان می‌دهد که:

- اکثر واحدها در محدوده **قیمت منصفانه (normal)** قرار دارند، با متراژ میانگین نزدیک به هم ولی قیمت نزدیک به سطح تعادلی بازار.
- واحدهای **گران (over-value)** متراژی مشابه ولی قیمت به مراتب بالاتر از سطح تعادلی دارند و در چند محله مشخص (مثل پونک و چیتگر-لیک) تمرکز بیشتری نشان می‌دهند، که می‌تواند برای شناسایی فرصت‌های مذاکره یا هشدار به خریداران استفاده شود
- واحدهای **ارزان (under-value)** با متراژی مشابه اما قیمت بسیار پایین‌تر از سطح مدل شناسایی شده‌اند و می‌توانند به عنوان فرصت خرید سرمایه‌گذاری یا فایل‌های مشکوک به خطای ثبت قیمت بررسی شوند

۷. دسته‌بندی (classification) با استفاده از مقدارهای متنی

هدف اصلی این بخش، بارگذاری داده‌های خام، انجام بررسی‌های اولیه، و سپس تمرکز بر روی پاکسازی و پیش‌پردازش داده‌های متنی (توصیف و عنوان آگهی‌ها) به منظور آماده‌سازی آن‌ها برای مراحل بعدی تحلیل و مدل‌سازی پردازش زبان طبیعی (NLP) است. با توجه به حجم و تنوع بالای داده‌ها، تمرکز کار بر روی **فیلترسازی**

هوشمند و استانداردسازی عمیق متون فارسی بود تا داده‌ای با کیفیت و مرتبط برای استخراج ویژگی‌های متنی در اختیار مدل قرار گیرد.

الف – آماده سازی مجموعه داده

پاکسازی متون و آماده‌سازی اولیه

به منظور بهره‌برداری کامل از اطلاعات متنی موجود در آگهی‌ها، دو ستون description (توصیف) و title (عنوان) با یکدیگر ادغام شدند تا ستون not_cleaned ایجاد شود. این تجمیع اطمینان حاصل می‌کند که تمام اطلاعات متنی موجود برای تحلیل استفاده شود. سپس فرآیند پاکسازی آغاز شد که شامل چندین مرحله حیاتی بود. ابتدا، یک تابع پاکسازی سخت‌گیرانه طراحی شد که وظیفه حذف کلیه کاراکترهای غیرفارسی، اموجی، کلمات انگلیسی و علائم نگارشی را از متن و همچنین حفظ کردن اعداد داخل متن (به‌علت آنکه بعضی از توضیحات در آن برای متراژ از عدد مثلاً ۱۰۰ متر استفاده شده است) بر عهده داشت. منطق این حذف، تمرکز صرف بر روی کلمات کلیدی فارسی بود تا از ورود نویزهای عددی و غیرمرتبط جلوگیری شود. علاوه بر این، جهت رفع مشکلات مربوط به قالب‌بندی (مانند خطوط جدید و فاصله‌های اضافی)، تمامی کاراکترهای \n, \r, و \t با یک فاصله استاندارد جایگزین شدند و فواصل اضافی نیز حذف گردیدند. این نتایج در ستون نهایی clean_text ذخیره شدند. پس از تولید این ستون، ستون‌های اصلی و میانی موقت (description, title, not_cleaned, not_cleaned2) حذف شدند تا حجم حافظه اشغالی کاهش یابد و تنها متون پاکسازی شده حفظ شوند.

پیش‌پردازش پیشرفته متون فارسی

مهم‌ترین عملیات این بخش، اعمال تکنیک‌های پردازش زبان طبیعی (NLP) با استفاده از کتابخانه استاندارد Hazm بر روی ستون clean_text بود. این مراحل برای تبدیل متن خام به فرمت قابل استفاده برای الگوریتم‌های یادگیری ماشین ضروری هستند:

۱. نرمال‌سازی (Normalization): در گام اول، تابع normalizer.normalize از کتابخانه Hazm

اعمال شد. این کار برای استانداردسازی نویسه‌های فارسی (مانند تبدیل "ی" عربی به "ی" فارسی و "ک" "

عربی به "ک" فارسی) انجام گرفت که برای یکپارچگی واژگان و جلوگیری از تولید واژه‌های تکراری با املای متفاوت، بسیار حیاتی است.

۲. **توکن‌سازی (Tokenization):** پس از نرمال‌سازی، متن به واحدهای مجزا به نام **توکن (Token)** تبدیل شد. این فرآیند با استفاده از `tokenizer.tokenize()` انجام و نتایج در ستون `tokens` ذخیره شد. این گام، ساختار متنی را برای تحلیل واژه‌ای آماده می‌سازد.

۳. **ریشه‌یابی (Stemming):** در این مرحله، از کلاس `Stemmer` کتابخانه `Hazm` استفاده شد تا هر توکن به ریشه (`Stem`) خود تبدیل شود. هدف از ریشه‌یابی، کاهش تنوع واژگانی با تبدیل اشکال صرفی و اشتقاقی یک کلمه به یک فرم واحد است (مثلاً تبدیل "طبقات" و "طبقه‌ها" به تک ریشه "طبقه"). این کار به مدل اجازه می‌دهد تا مفاهیم مرتبط را به صورت تجمیعی درک کند و قدرت تعمیم آن افزایش یابد.

۴. **حذف کلمات توقف (Stopword Removal):** در نهایت، برای افزایش نسبت سیگنال به نویز در داده‌های متنی، کلمات توقف (مانند حروف ربط و اضافه که ارزش معنایی کمی دارند) حذف شدند. این کلمات از یک فایل بیرونی به نام `stopwords` بارگذاری و به صورت یک مجموعه (`set`) برای جستجوی سریع در حافظه نگهداری شدند. پس از فیلتر کردن توکن‌های ریشه‌یابی شده بر اساس این لیست، نتایج نهایی در ستون `clean_text1` ذخیره گردید.

آماده‌سازی نهایی و خروجی داده‌ها

پس از تکمیل کلیه مراحل پاکسازی و پیش‌پردازش متنی، ستون `clean_text` که حاوی لیست ریشه‌های پاکسازی شده بود، به فرمت رشته متنی تبدیل شد (با استفاده از تابع `join`). این ساختار رشته‌ای، فرمت استاندارد مورد نیاز برای تکنیک‌های برداری‌سازی (`Vectorization`) مانند `TF-IDF` است که در مراحل بعدی مدل‌سازی استفاده خواهند شد.

در گام نهایی، یک `DataFrame` جدید با نام `df_final` ایجاد شد که تنها شامل ستون‌های ضروری `cat3_slug` (متغیر هدف)، `user_type` (متغیر ویژگی) و `clean_text` (ویژگی متنی نهایی) بود. این مجموعه داده تمیز و آماده به عنوان ورودی برای بخش بعدی که به استخراج ویژگی و مدل‌سازی اختصاص دارند، مورد استفاده قرار گیرد.

ب - مدل‌سازی و دسته‌بندی

این مرحله از پروژه، بر مدل‌سازی دسته‌بندی متون پاکسازی شده متمرکز است. هدف اصلی، دسته‌بندی آگهی‌ها بر اساس متن توصیف آن‌ها به ۱۶ دسته‌بندی انواع ملک موجود در ستون `cat3_slug` و همچنین پیش‌بینی نوع کاربری آگهی‌دهنده برای داده‌های گمشده است. برای دستیابی به این اهداف، چندین تکنیک استخراج ویژگی و مدل‌های یادگیری ماشین کلاسیک و یادگیری عمیق مورد بررسی و ارزیابی قرار گرفتند.

بارگذاری داده و بررسی اولیه

عملیات بر روی خروجی مرحله قبل که حاوی متون پاکسازی، ریشه‌یابی و توکن‌سازی شده در ستون `clean_text` بود انجام شد. این مجموعه داده دارای تقریباً یک میلیون رکورد (۹۹۹,۹۴۵) است که در ستون `cat3_slug` (متغیر هدف اصلی) دارای ۱۶ کلاس مختلف است. ستون `user_type` که برای هدف ثانویه پیش‌بینی مورد نیاز است، حاوی درصد بالایی از مقادیر گمشده بود (تنها ۲۸۸,۸۷۷ رکورد غیرتهی از ۹۹۹,۹۴۵ رکورد). با توجه به حجم بالای داده‌ها و محدودیت‌های محاسباتی و سخت‌افزاری، یک گام حیاتی نمونه‌برداری انجام شد؛ بدین صورت که ۴۰ درصد از داده‌ها به صورت تصادفی انتخاب شدند تا تحلیل‌های اولیه و مدل‌سازی آزمایشی با سرعت و کارایی بیشتری انجام پذیرد.

استخراج ویژگی متنی و برداری‌سازی

در این مرحله، متون تمیز به فرمت‌های عددی قابل درک برای مدل‌های یادگیری ماشین تبدیل شدند. سه رویکرد اصلی برای برداری‌سازی پیاده‌سازی و مقایسه شدند:

مدل‌های مبتنی بر شمارش:

- **BagOfWords**: از `CountVectorizer` استفاده شد تا فراوانی هر واژه در هر سند ثبت شود.
- **TF-IDF**: از `TfidfVectorizer` استفاده شد که نه تنها فراوانی واژه را در سند، بلکه اهمیت آن را در کل مجموعه (با کاهش وزن کلمات پرتکرار و کم‌ارزش) نیز در نظر می‌گیرد.
- در هر دو روش، تعداد ویژگی‌های استخراجی به ۱۰,۰۰۰ محدود شد تا ابعاد مجموعه داده کاهش یابد و از ورود کلمات بسیار نادر به واژه‌نامه جلوگیری شود.

مدل مبتنی بر تعبیه (Embedding) و توالی:

- **Word2Vec**: یک مدل Word2Vec با استراتژی Skip-gram، ابعاد برداری ۱۰۰، و min_count (حداقل فراوانی کلمه) برابر با ۲ آموزش داده شد. هدف از این روش، درک مفاهیم معنایی کلمات و روابط آن‌ها بود. برای تبدیل بردار کلمات به بردار سند، از میانگین بردارهای کلمات موجود در هر سند استفاده شد، به طوری که هر آگهی توسط میانگین تعبیه‌های واژگان خود نمایش داده گردد.

در رویکرد یادگیری عمیق، به جای استفاده مستقیم از BoW یا TF-IDF، از Tokenizer و pad_sequence برای تبدیل متن‌ها به توالی‌های عددی با طول ثابت استفاده شد؛ در این روش، هر واژه به یک شناسه عددی نگاشت شد و متن هر آگهی در قالب دنباله‌ای از این شناسه‌ها به مدل LSTM داده شد.

تقسیم داده و آماده‌سازی برای آموزش

برای ارزیابی منصفانه و دقیق عملکرد مدل‌ها، به دلیل محدودیت‌های سخت‌افزاری و زمانی امکان آموزش همه مدل‌ها روی کل حدود یک میلیون رکورد وجود نداشت. بنابراین ابتدا یک نمونه‌ی تصادفی شامل ۴۰ درصد از کل داده‌ها انتخاب شد و تمام مدل‌ها (Word2Vec، Naive Bayes، Random Forest، Logistic Regression) و LSTM فقط روی همین زیرمجموعه آموزش و ارزیابی شدند. سپس برای مدل LSTM، داده‌های توالی‌سازی‌شده‌ی همین ۴۰ درصد با استفاده از تابع train_test_split به دو بخش آموزش و آزمون با نسبت ۸۰ به ۲۰ و random_state=42 تقسیم شد تا ارزیابی نهایی روی داده‌های دیده‌نشده نیز انجام شود.

مدل‌سازی دسته‌بندی هدف (cat3_slug – نوع ملک)

برای مقایسه عملکرد رویکردهای مختلف، مجموعه داده نمونه‌برداری شده (که با BoW و TF-IDF برداری شده بود) به نسبت ۸۰ درصد برای مجموعه آموزش و ۲۰ درصد برای مجموعه تست تقسیم شد. سه مدل دسته‌بندی بر روی این ویژگی‌ها آموزش داده شدند:

رگرسیون لجستیک: به عنوان یک مدل خطی کارآمد و قوی در دسته‌بندی متنی، بر روی هر دو نمایش BoW و TF-IDF آموزش داده شد.

جنگل تصادفی: این مدل ابتدا با پارامترهای سریع و سطحی (عمق ۱۰، ۲۰ درخت) برای تعیین یک خط پایه سریع آموزش داده شد و سپس با پارامترهای بهینه‌سازی شده (مانند $\text{max_depth}=20$, $\text{n_estimators}=50$) و مهم‌تر از همه، تنظیم وزن کلاس‌ها ($\text{class_weight}='balanced'$) برای مقابله با عدم توازن ذاتی کلاس‌ها، تنظیم شد.

بیز ساده چندجمله‌ای: این مدل به دلیل کارایی بالا در مسائل دسته‌بندی مبتنی بر فراوانی (مانند BoW و TF-IDF)، به عنوان یک معیار پایه‌ی قوی ارزیابی شد.

برای ارزیابی کمی، از تابع `eval_model` استفاده شد که با بهره‌گیری از `accuracy_score` و `f1_score` (با `'average'='weighted'`)، برای هر ترکیب «مدل - نمایش ویژگی» مقادیر Accuracy و Weighted F1 را محاسبه و گزارش می‌کند. نتایج نهایی به صورت زیر به دست آمد:

- LogReg (BoW): Accuracy = 0.825 ، Weighted F1 = 0.823
- RandomForest (BoW): Accuracy = 0.495 ، Weighted F1 = 0.410
- RandomForest+ (BoW): Accuracy = 0.698 ، Weighted F1 = 0.716
- NaiveBayes (BoW): Accuracy = 0.749 ، Weighted F1 = 0.755
- LogReg (TF-IDF): Accuracy = 0.829 ، Weighted F1 = 0.826
- RandomForest (TF-IDF): Accuracy = 0.494 ، Weighted F1 = 0.413
- RandomForest+ (TF-IDF): Accuracy = 0.698 ، Weighted F1 = 0.716
- NaiveBayes (TF-IDF): Accuracy = 0.729 ، Weighted F1 = 0.715

این نتایج نشان داد که در این کار دسته‌بندی متنی، نمایش TF-IDF در مجموع نسبت به BoW برای رگرسیون لجستیک و نسخه‌ی تنظیم‌شده جنگل تصادفی عملکرد اندکی بهتری دارد، در حالی که برای Naive Bayes، نمایش BoW کمی مناسب‌تر است. در میان تمام مدل‌های کلاسیک، Logistic Regression همراه با ویژگی‌های TF-IDF بهترین عملکرد را ارائه داد ($\text{Accuracy} \approx 0.829$ و $\text{Weighted F1} \approx 0.826$) و به عنوان baseline کلاسیک مرجع برای مسئله‌ی دسته‌بندی `cat3_slug` در نظر گرفته شد. در مقابل، رویکرد Word2Vec با میانگین‌گیری تعبیه‌ها با وجود پیچیدگی بیشتر، نتایج ضعیف‌تری نسبت به TF-IDF به دست آورد (به‌طور نمونه

Accuracy در حدود ۰,۷۷۸ و Weighted F1 در حدود ۰,۷۷۴، که می‌تواند ناشی از دست رفتن اطلاعات ترتیبی و ساختاری جملات در فرآیند میانگین‌گیری باشد.

کامپایل و پیکربندی مدل LSTM

در گام بعد، برای بهره‌گیری از اطلاعات ترتیبی و معنایی دقیق‌تر در متن، یک مدل یادگیری عمیق مبتنی بر LSTM برای دسته‌بندی cat3_slug طراحی شد. پس از تعریف معماری، مدل با استفاده از متد compile پیکربندی شد. در این مرحله، تابع زیان مناسب برای مسائل دسته‌بندی چندکلاسه انتخاب شد (sparse_categorical_crossentropy با توجه به کدگذاری عددی y)، به عنوان بهینه‌ساز از Adam استفاده شد که یک الگوریتم بهینه‌سازی مبتنی بر گرادینت تطبیقی است و برای مسائل با ابعاد بالا و داده‌های متنی عملکرد بسیار خوبی دارد، و معیار ارزیابی نیز accuracy تعیین شد تا در طول آموزش و ارزیابی، دقت مدل به صورت درصد نمونه‌های طبقه‌بندی شده صحیح محاسبه و گزارش شود. انتخاب این پارامترها با توجه به ماهیت مسئله دسته‌بندی متنی و حجم نسبتاً بزرگ داده‌ها کاملاً منطقی و استاندارد است.

آموزش مدل و نظارت بر فرآیند یادگیری

آموزش مدل با فراخوانی متد fit بر روی داده‌های آموزشی X_train و y_train انجام شد. تعداد epoch برابر با ۱۰ تعیین شد که به معنای عبور کامل مدل از روی تمام داده‌های آموزشی به تعداد ۱۰ بار است. اندازه batch_size برابر با ۳۲ انتخاب شد، بدین معنی که در هر مرحله از به‌روزرسانی وزن‌ها، مدل روی ۳۲ نمونه به طور همزمان پردازش و گرادینت محاسبه می‌کند که تعادل مناسبی بین سرعت آموزش و پایداری همگرایی ایجاد می‌کند. همچنین پارامتر validation_data برابر با (X_test, y_test) تنظیم شد تا در پایان هر epoch، عملکرد مدل بر روی مجموعه آزمون نیز محاسبه و گزارش شود و امکان پایش روند یادگیری و تشخیص بیش‌برازش در حین آموزش فراهم گردد.

تحلیل نتایج آموزش و ارزیابی عملکرد

لاگ‌های خروجی آموزش اطلاعات ارزشمندی درباره روند یادگیری مدل ارائه می‌دهند. در epoch اول، مدل با accuracy آموزشی حدود ۰,۷۳ و val_accuracy حدود ۰,۸۳ شروع کرد که نشان می‌دهد مدل حتی در ابتدای آموزش توانسته الگوهای قابل توجهی از داده‌ها را یاد بگیرد. همچنین loss آموزشی در حدود ۰,۸۰ و val_loss

در حدود ۰,۴۷ بود. با پیشرفت آموزش، accuracy مجموعه آموزش به تدریج افزایش یافت و در epoch دهم به حدود ۰,۸۹ رسید، در حالی که loss آموزشی به حدود ۰,۲۸ کاهش پیدا کرد.

با این حال، در عملکرد روی مجموعه اعتبارسنجی، val_accuracy در epoch دوم و سوم به حدود ۰,۸۴ رسید و پس از آن تا پایان آموزش در محدوده ۰,۸۳ تا ۰,۸۴ نوسان کرد و بهبود قابل توجهی نداشت. مهم‌تر از آن، val_loss از epoch چهارم به بعد روند صعودی پیدا کرد و از حدود ۰,۴۴ در epoch سوم به حدود ۰,۵۴ در epoch دهم افزایش یافت. این الگو نشانه‌ای واضح از بیش‌برازش است، به این معنی که مدل در حال حفظ کردن جزئیات خاص داده‌های آموزشی است و قابلیت تعمیم آن روی داده‌های جدید در حال کاهش می‌باشد. بر اساس این نتایج، می‌توان نتیجه گرفت که حدود ۳ تا ۵ epoch برای این معماری و این حجم از داده کفایت می‌کند و ادامه آموزش فراتر از این مقدار تنها به افزایش هزینه محاسباتی و تشدید بیش‌برازش منجر خواهد شد.

نتایج نهایی و دستاوردهای مدل

با وجود نشانه‌های بیش‌برازش در epoch‌های پایانی، مدل LSTM توانست به دقت قابل توجه حدود ۰,۸۴ روی مجموعه آزمون دست یابد که برای یک مسئله دسته‌بندی چند کلاسه با ۱۶ دسته و داده‌های متنی غیر ساخت یافته بسیار رضایت بخش است. این نتیجه نشان می‌دهد که معماری LSTM همراه با لایه Embedding توانسته به خوبی الگوهای معنایی و ترتیبی موجود در توضیحات آگهی‌ها را یاد بگیرد و بر اساس آن‌ها نوع ملک را با دقت بالا تشخیص دهد؛ مقایسه این عملکرد با دقت تصادفی (حدود ۶,۲۵ درصد برای ۱۶ کلاس) اهمیت دستاورد مدل را نشان می‌دهد. در کنار این مدل، نتایج مدل‌های کلاسیک نیز تصویر کاملی از فضای خط مبنا ارائه کرده‌اند و نشان داده‌اند که Logistic Regression + TF-IDF قوی‌ترین baseline کلاسیک است، در حالی که LSTM بر پایه توالی توانسته با استفاده از ساختار جملات به دقت بالاتری دست یابد.

پیش‌بینی نوع کاربری آگهی‌دهنده (user_type)

با توجه به حجم بالای داده‌های گمشده در ستون user_type (تنها ۲۸۸,۸۷۷ رکورد از ۹۹۹,۹۴۵ رکورد دارای مقدار غیر تهی بودند)، یک زیر پروژه برای جایگزینی این مقادیر با استفاده از متن آگهی‌ها تعریف شد. این فرآیند برای غنی‌سازی مجموعه داده اصلی با یک ویژگی مهم برای مدل‌های آتی ضروری بود.

شناسایی عدم توازن کلاس‌ها

بررسی اولیه توزیع `user_type` نشان داد که عدم توازن شدیدی بین دو کلاس وجود دارد: کلاس "مشاور املاک" با ۱۰۲,۰۴۳ نمونه به عنوان کلاس اکثریت و کلاس "شخصی" با تنها ۱۳,۳۴۸ نمونه به عنوان کلاس اقلیت شناسایی شدند. این نسبت نابرابر (حدود ۷,۶ به ۱) می‌توانست باعث شود مدل به سادگی همه نمونه‌ها را به کلاس اکثریت نسبت دهد و در تشخیص کلاس اقلیت ضعیف عمل کند.

مدیریت عدم توازن با وزن‌دهی کلاس

برای مقابله با این مشکل، از تکنیک وزن‌دهی کلاس (`class weighting`) استفاده شد. با استفاده از تابع `compute_class_weight` از کتابخانه `scikit-learn`، وزن‌های مناسب برای هر کلاس محاسبه شد: کلاس اقلیت (شخصی) وزن ۴,۳۲ و کلاس اکثریت (مشاور املاک) وزن ۰,۵۷ دریافت کرد. این وزن‌ها در طول آموزش مدل به تابع زیان اعمال شدند تا خطای طبقه‌بندی نادرست نمونه‌های کلاس اقلیت با شدت بیشتری جریمه شود و مدل مجبور باشد برای هر دو کلاس الگوهای معنادار یاد بگیرد.

معماری مدل LSTM برای `user_type`

برای این دسته‌بندی دو کلاسه، یک شبکه LSTM مشابه مدل `cat3_slug` طراحی شد که شامل لایه `Embedding`، لایه LSTM، لایه `Dense` میانی با `Dropout` و لایه خروجی با تابع فعال‌سازی `sigmoid` (مناسب برای طبقه‌بندی دو کلاسه) بود. تابع زیان `binary_crossentropy` و بهینه‌ساز `Adam` انتخاب شد و پارامتر `class_weight` در متد `fit` تعیین شد تا وزن‌های محاسبه‌شده در فرآیند آموزش لحاظ شوند.

آموزش و نتایج

مدل با `epoch=5`، `batch_size=۳۲` و `validation_split=۰,۲` آموزش داده شد. لاگ‌های آموزش نشان دادند که مدل در `epoch` اول با `accuracy` آموزشی ۰,۸۲ و `val_accuracy` حدود ۰,۹۰ شروع کرد و تا `epoch` دوم به `val_accuracy` حدود ۰,۹۱ رسید. پس از آن، `val_loss` اندکی نوسان داشت اما مدل به طور کلی پایدار باقی ماند و نشانه‌های بیش‌برازش شدید مشاهده نشد.

ارزیابی نهایی روی مجموعه آزمون نشان داد که مدل LSTM با وزن‌دهی کلاس به `accuracy` کلی ۰,۸۹ (۸۹ درصد) و `weighted F1-score` برابر ۰,۹۰ دست یافت. مهم‌تر از آن، `precision` و `recall` برای کلاس اقلیت ("شخصی") به ترتیب ۰,۵۳ و ۰,۸۱ بود که نشان می‌دهد مدل با وجود عدم توازن شدید، توانسته ۸۱ درصد از نمونه‌های

کلاس شخصی را به درستی تشخیص دهد. برای کلاس اکثریت ("مشاور املاک")، precision و recall به ترتیب ۰,۹۷ و ۰,۹۱ به دست آمد که عملکرد بسیار قوی این مدل را نشان می دهد.

جایگزینی مقادیر گم شده

پس از آموزش و اعتبارسنجی مدل، از آن برای پیش بینی user_type برای بیش از ۷۰۰,۰۰۰ رکورد فاقد برچسب (null) استفاده شد. مقادیر پیش بینی شده در یک ستون جدید به نام predicted_user_type ذخیره شدند تا مجموعه داده اصلی دست نخورده باقی بماند و امکان مقایسه و بررسی بیشتر فراهم شود. این فرآیند موفقیت آمیز نشان دهنده توانایی مدل LSTM در استفاده از محتوای متنی آگهی ها برای استنتاج نوع کاربری آگهی دهنده است و مجموعه داده را برای مراحل بعدی تحلیل و مدل سازی غنی تر کرده است.

۸. نتیجه گیری

پروژه حاضر با هدف تحلیل جامع آگهی های ملکی تهران که از پلتفرم دیوار جمع آوری شده اند، و با تمرکز بر استخراج الگوهای پنهان در بازار مسکن و ساخت مدل های قابل اعتماد برای پیش بینی قیمت و دسته بندی داده ها، طراحی و اجرا گردید. مسیر تحلیل نشان داد که داده های خام، چه ساختاری و چه متنی، بدون استانداردسازی عمیق، قابلیت استفاده در مدل های علمی را ندارند.

اولین و حیاتی ترین دستاورد پروژه، ایجاد یک مجموعه داده منسجم و قابل اتکا بود. در این مرحله، تکرارهای مفهومی حذف، و مقادیر پرت (Outliers) غیر واقعی مدیریت شدند. برای مقابله با چولگی شدید متغیرهای مالی و ابعادی (مانند قیمت و متراژ)، از تبدیل لگاریتمی و روش IQR با اعمال فیلتر مبتنی بر نوع ملک استفاده شد تا داده های پرت به صورت علمی حذف گردند. همچنین، مقادیر گم شده ستون های کلیدی (مانند متراژ و تعداد اتاق) با استفاده از روش های Group-wise Imputation و بر اساس همبستگی با ویژگی های مرتبط پر شدند تا زمینه اطلاعات حفظ شود. این پایه گذاری دقیق، پایداری و معناداری نتایج مدل سازی های بعدی را تضمین کرد.

با هدف کشف ساختار ذاتی بازار املاک، عملیات خوشه بندی (K-Means) بر روی ۸۳۳ هزار ملک انجام شد. با انتخاب هوشمندانه ویژگی ها و استانداردسازی داده ها، تحلیل کاهش ابعاد با استفاده از PCA نشان داد که ۹۰,۲۲٪ واریانس داده ها با ۸ مؤلفه اصلی پوشش داده می شود. در نهایت، یک ساختار پایدار با ۸ خوشه اصلی شناسایی شد که پایداری آن ها بین ۹۶٪ تا ۱۰۰٪ ارزیابی گردید. این نتایج تأیید می کند که الگوهای قیمتی، ابعادی و فیزیکی

ملک‌ها واقعاً دارای ساختاری چندگروهی و معنادار هستند و بازار مسکن را می‌توان به بخش‌های مجزا و قابل تعریف تقسیم کرد.

در این بخش، یک مدل رگرسیونی برای پیش‌بینی قیمت ملک‌ها ساخته شد. مهم‌ترین مهندسی ویژگی، محاسبه میانگین قیمت محله (با استفاده از میانه برای کاهش اثر پرت‌ها) بود که تأثیر مستقیمی بر بهبود عملکرد مدل داشت. در ارزیابی مدل‌های مختلف (Linear Regression، KNN، Decision Tree و MLP)، مدل‌های Decision Tree و KNN پس از بهینه‌سازی ابرپارامترها، بهترین عملکرد را ارائه دادند. مدل منتخب توانست قیمت را با دقت قابل‌قبولی تخمین زده و در نهایت، املاک را از نظر قیمت‌گذاری به سه دسته کلیدی تقسیم کند: گران‌تر از ارزش (Over-value)، منصفانه (Normal) و ارزان‌تر از ارزش (Under-value). این دسته‌بندی امکان شناسایی املاک گران‌فروشی یا فرصت‌های سرمایه‌گذاری را فراهم می‌کند.

در بخش پردازش زبان طبیعی، متون آگهی‌ها (عنوان و توضیحات) تحت یک فرآیند پاکسازی عمیق فارسی (توکن‌سازی، ریشه‌یابی با Hazm و حذف توقف‌واژه‌ها) قرار گرفتند.

- دسته‌بندی نوع ملک (cat3_slug): در این وظیفه، مدل Logistic Regression با ویژگی‌های TF-IDF بهترین عملکرد را در میان مدل‌های کلاسیک ارائه داد و به دقت (Accuracy) ۸۲,۹٪ و Weighted F1-score ۸۲,۶٪ دست یافت.

- پیش‌بینی نوع آگهی‌دهنده (user_type): یک شبکه LSTM برای پیش‌بینی اینکه آگهی‌دهنده "شخصی" است یا "مشاور املاک"، با موفقیت آموزش داده شد. این مدل به دقت کلی ۸۹٪ رسید و مهم‌تر از آن، علی‌رغم عدم توازن شدید داده‌ها، Recall ۸۱٪ را برای کلاس اقلیت ("شخصی") کسب کرد. این دقت بالا به ما امکان داد تا مقادیر گم‌شده این ویژگی را برای بیش از ۷۰۰ هزار رکورد با اطمینان تکمیل کنیم.

این پروژه نشان داد که با ترکیب روش‌های دقیق پاکسازی داده‌های ساختاری و تکنیک‌های پیشرفته یادگیری ماشین (Clustering, Regression, Classification) در کنار پردازش زبان طبیعی فارسی (NLP)، می‌توان از داده‌های آگهی‌های آنلاین، ارزش تجاری و تحلیلی بالایی استخراج کرد. دستاوردهای این پروژه، زیربنای مناسبی برای توسعه ابزارهای هوشمندی همچون: سیستم‌های قیمت‌گذاری خودکار (AVM)، ابزارهای شناسایی ناهنجاری‌ها و قیمت‌گذاری‌های غیرواقعی، و سیستم‌های پیشنهادی بهینه املاک است که می‌تواند مستقیماً به شفافیت و کارایی بیشتر در بازار مسکن کمک کند.